

Project Objective: Personal Cloud Hub

Overview

Personal Cloud Hub is a desktop application that automatically monitors selected folders on a computer, organizes files, uploads them to Google Drive, maintains a searchable local database, and provides a professional dashboard for managing personal documents, coding files, images, projects, certificates, and study materials.

The goal is to eliminate manual file management and create a centralized personal storage system.

Problem Statement

Students and professionals often face problems such as:

- Files scattered across multiple folders
- Difficulty finding old documents
- Forgotten backups
- Duplicate files
- Unorganized Google Drive storage
- Loss of important files due to system failure
- No centralized view of uploaded files
- Manual upload process consuming time

Example:

Desktop
Downloads
Documents
Projects
College Notes
Certificates
External Drive
Google Drive

Files are spread everywhere and become difficult to manage over time.

Primary Objective

To create a desktop application that automatically:

1. Detects new files.
2. Organizes files intelligently.
3. Uploads files to Google Drive.
4. Tracks uploaded files in a local database.
5. Provides a dashboard for monitoring storage and activity.
6. Enables quick search and retrieval.
7. Maintains a personal cloud archive.

Functional Objectives

1. Automatic File Monitoring

The system continuously watches selected folders.

Example:

C:\Users\Aryan\Uploads

When a file is added:

Resume.pdf

the application automatically detects it.

Goal

Remove the need for manual uploads.

2. Automated Cloud Backup

Detected files are uploaded to Google Drive.

Goal

Ensure important files are backed up automatically.

Benefits:

- Protection against data loss
- Device-independent access
- Reduced manual effort

3. Intelligent File Organization

Files are categorized automatically.

Example:

Resume.pdf



Documents

project.py



Coding

certificate.jpg



Certificates

Goal

Maintain a structured cloud storage system.

4. Centralized Dashboard

Provide a single interface for monitoring:

- Total files
- Upload status
- Storage usage
- Recent activity
- Upload history

Goal

Give complete visibility into file management operations.

5. Search and Retrieval

Users can search files instantly.

Example:

resume

Results:

Resume.pdf

Resume_2025.pdf

Goal

Reduce time spent locating files.

6. Upload History Tracking

Maintain records of:

- File names
- Upload dates
- Upload status
- Cloud location

Goal

Provide accountability and traceability.

7. Duplicate Prevention

Prevent repeated uploads of the same file.

Goal

Reduce unnecessary storage consumption.

8. Analytics and Insights

Generate statistics such as:

- Files uploaded per day
- Most common file types
- Storage consumption
- Upload trends

Goal

Help users understand storage usage patterns.

Technical Objectives

Database Management

Use SQLite to store metadata.

Store:

Filename

Path

Type

Size

Upload Date

Drive ID

Status

Goal

Enable efficient search, analytics, and tracking.

Google Drive Integration

Connect securely with Google Drive API.

Goal

Create seamless cloud synchronization.

Real-Time Monitoring

Use Watchdog to monitor file-system events.

Goal

Provide instant response to new files.

Professional User Interface

Build a modern desktop application using PyQt6.

Goal

Deliver an intuitive and efficient user experience.

User Objectives

For Students

Manage:

- Notes
- Assignments
- Research papers
- Presentations
- Certificates
- Project files

For Developers

Manage:

- Source code
- Git archives
- Documentation
- Datasets
- Technical notes

For Professionals

Manage:

- Reports
- Contracts
- Presentations
- Office documents
- Personal records

Expected Outcomes

After implementation, the user should be able to:

Automatically Upload Files

Add File



Automatic Upload

Search Files Instantly

Search



Results

View Storage Analytics

Total Files

Storage Usage

Upload Trends

Access Files Anywhere

Local System



Google Drive



Any Device

Future Scope

Potential future enhancements:

- OCR-based document indexing
- AI-powered file classification
- Natural language search
- Multi-cloud support (Google Drive, OneDrive, Dropbox)
- Mobile companion app
- Encrypted file storage

- Version control system
- Shared workspace support

Final Objective Statement

The objective of Personal Cloud Hub is to build a centralized personal file-management and cloud-backup platform that automatically monitors local folders, organizes files, synchronizes them with Google Drive, maintains searchable metadata in a local database, and provides a professional dashboard for storage management, analytics, and file retrieval, thereby eliminating manual file organization and backup tasks.

This project is large enough that you should build it in stages. Don't start with the dashboard. Start with the foundation.

Final Architecture

Personal Cloud Hub

Desktop Folder

|



File Monitor (Watchdog)

|



Upload Manager

|



|



SQLite Database

|



PyQt6 Dashboard

Phase 1 — Environment Setup

Install Python

Install Python 3.10 +

Check:

`python --version`

Create Project

```
mkdir PersonalCloudHub
```

```
cd PersonalCloudHub
```

Create Virtual Environment

```
python -m venv venv
```

Activate:

Windows

```
venv\Scripts\activate
```

Install Libraries

```
pip install pyqt6
```

```
pip install watchdog
```

```
pip install pydrive2
```

```
pip install sqlite-utils
```

```
pip install plyer
```

```
pip install pyqtgraph
```

```
pip install python-dotenv
```

Phase 2 — Project Structure

PersonalCloudHub

```
├— app.py
```

```
├— config
```

```
|   └— settings.json
```

```
|   └— credentials.json
```

```
├— database
```

- | └─ files.db

- | └─ services

- | | └─ drive_service.py

- | | └─ monitor_service.py

- | | └─ database_service.py

- | | └─ upload_service.py

- | └─ notification_service.py

- | └─ ui

- | | └─ dashboard.py

- | | └─ upload_page.py

- | | └─ search_page.py

- | | └─ analytics_page.py

- | └─ settings_page.py

- | └─ assets

- | | └─ icons

- | | └─ images

- └─ logs

Phase 3 — Create Google Drive API

You need a Google Cloud project.

Go to:

[Google Cloud Console](#)

Create Project

Personal Cloud Hub

Enable API

Enable:

Google Drive API

Create OAuth Credentials

Create:

OAuth Client ID

Application Type:

Desktop Application

Download:

credentials.json

Place inside:

config/

Phase 4 — Connect Google Drive

First launch:

App Starts

↓

Google Login Window Opens

↓

Allow Access

↓

Token Saved

Token:

token.json

Saved locally.

Future launches:

No Login Required

Phase 5 — Create Database

Create:

database/files.db

Files Table

```
CREATE TABLE files (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    filename TEXT,  
    filepath TEXT,  
    filesize INTEGER,  
    filetype TEXT,  
    upload_date TEXT,  
    google_drive_id TEXT,  
    status TEXT  
);
```

Settings Table

```
CREATE TABLE settings (  
    id INTEGER PRIMARY KEY,  
    key TEXT,  
    value TEXT  
);
```

Upload Logs Table

```
CREATE TABLE upload_logs (  
    id INTEGER PRIMARY KEY,  
    filename TEXT,  
    upload_time TEXT,  
    message TEXT  
);
```

Phase 6 — Folder Monitoring

User selects:

C:\Users\Aryan\Uploads

Watchdog watches folder.

When file added:

Resume.pdf

Event triggers:

Upload Manager

Phase 7 — Upload Flow

File Detected

↓

Validate File

↓

Check Duplicate

↓

Upload to Drive

↓

Get Drive ID

↓

Save to SQLite



Notification

Phase 8 — Auto Categorization

Google Drive Structure

My Drive

└— Documents

└— Coding

└— Images

└— Projects

└— Certificates

└— Archives

Rules:

pdf/docx/pptx



Documents

py/java/js



Coding

jpg/png/webp



Images

zip/rar



Archives

Phase 9 — Dashboard UI

Main Window

Personal Cloud Hub	
Sidebar	
Dashboard	
Upload Center	
Search	
Analytics	
Settings	

Dashboard Cards

Total Files

Storage Used

Uploads Today

Failed Uploads

Google Drive Status

Recent Uploads

Resume.pdf

Java_Project.zip

DSA_Notes.pdf

Upload Queue

Resume.pdf

 100%

Search Page

Search:

resume

Query:

```
SELECT *
```

```
FROM files
```

```
WHERE filename
```

```
LIKE '%resume%';
```

Results shown instantly.

Analytics Page

Charts:

Uploads Per Day

Mon 15

Tue 23

Wed 12

Thu 40

File Types

PDF

Python

Images

ZIP

Use:

PyQtGraph

Notifications

Examples:

✓ Upload Successful

Resume.pdf

⚠ Upload Failed

Internet Connection Lost

Settings Page

Theme

Dark

Light

Upload Folder

Backup Folder

Auto Start

Notifications

Phase 10 — Advanced Features

After MVP works.

Add:

OCR

Extract text from:

PDF

PNG

JPG

Search inside images.

AI Search

Search:

Find my ML project report

instead of exact filename.

Duplicate Detection

Hash:

SHA256

Store hash in database.

File Preview

Preview:

PDF

TXT

Images

Code Files

Inside app.

Database Connection Flow

PyQt6 Dashboard

|



database_service.py

|



SQLite Database

Example:

```
get_total_files()
```

runs:

```
SELECT COUNT(*)
```

```
FROM files;
```

Returns:

2431

Dashboard card updates.

Google Drive Connection Flow

PyQt6

|



upload_service.py

|



drive_service.py

|



Google Drive API

Upload returns:

Google Drive File ID

Store in SQLite.

Final Build Order

1. Python setup
2. SQLite database
3. Google Drive authentication
4. Upload single file
5. Folder monitoring
6. Auto uploads
7. Dashboard UI
8. Search
9. Analytics
10. Notifications
11. Settings
12. Packaging with executable

Only after Steps 1–6 work should you start building the dashboard. Most people do the UI first and then struggle to connect everything later. Build the backend pipeline first, then connect it to the dashboard.